Data Management for Machine Learning — Assignment 2

End-to-End Recommendation System Pipeline Report

Student Name:	Rushikesh Sanjay Kothawade
Student ID:	2025AG05929
Course :	DMML Assignment 2 — MovieLens 100K
GitHub Repository:	https://github.com/RushikeshKothawade07/2025AG05929_DMML_Assignment2

1. Objective & Pipeline Overview

In this assignment, I built an end-to-end recommendation pipeline on the MovieLens 100K dataset (943 users, 1,682 items, and 100,000 interactions). The pipeline handles data ingestion, quality validation, time-based data splitting, transformation, feature generation, model training, offline evaluation, and workflow orchestration. The complete code is modularized inside 'src/', configured via 'configs/config.yaml', and executed through a master runner script.

2025AG05929

2. Pipeline Stages & Execution Screenshots

Stage 1: Data Ingestion (src/ingestion/ingest.py)

Downloads the raw MovieLens 100K dataset from GroupLens, unpacks it, and saves users, items, and ratings as CSV files in data/raw/. A total of 943 users, 1,682 items, and 100,006 raw rating interactions were ingested.

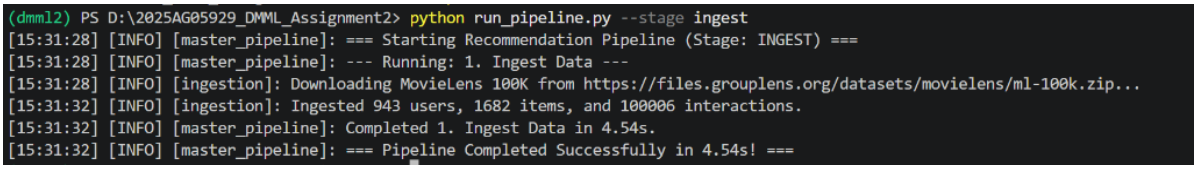


Figure 1: Stage 1 Ingestion output (python run_pipeline.py --stage ingest)

Stage 2: Data Quality Validation (src/validation/validate.py)

Runs automated checks for null values, invalid ratings (not between 1.0 and 5.0), corrupted timestamps, duplicate interactions, and foreign key referential integrity. Out of 100,006 raw records, 100,000 valid records were passed to data/processed/, while 6 invalid records were quarantined into data/errors/invalid_interactions.csv.

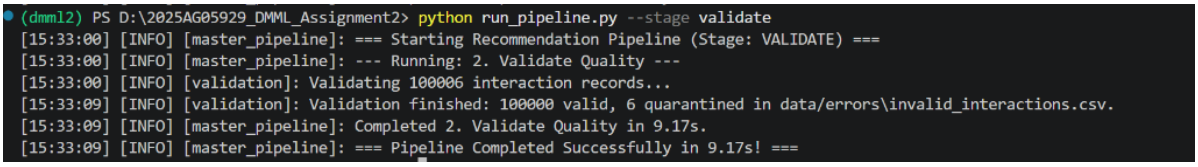


Figure 2: Stage 2 Validation output showing quarantine of 6 records (python run_pipeline.py --stage validate)

Validation Check	Rule	Status / Count
Missing Values	user_id, item_id, rating, timestamp not null	Passed (0 nulls)
Rating Domain	1.0 <= rating <= 5.0	Enforced (2 bad ratings quarantined)
Timestamp Bounds	Valid epoch timestamp (before 2030)	Enforced (2 bad timestamps quarantined)
Duplicate Rows	Unique (user_id, item_id, timestamp)	Enforced (2 duplicates quarantined)
Referential Integrity	Foreign key check against user & item tables	Passed (100% matched)

Stage 3: Data Preparation & Temporal Split (src/preparation/prepare.py)

Sorts interactions chronologically by timestamp and splits them into 80% training (80,000 records) and 20% test (20,000 records). Using a time-based split ensures future interactions do not leak into model training.

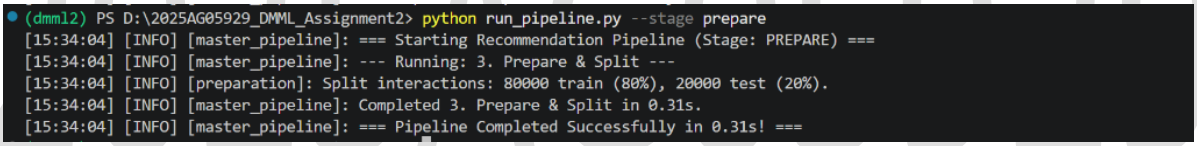


Figure 3: Stage 3 Preparation & 80/20 time split output (python run_pipeline.py --stage prepare)

Stage 4: Data Transformation & Sparse Matrix (src/transformation/transform.py)

Computes user activity counts, Bayesian-smoothed item popularity scores, and creates a Scipy CSR sparse user-item interaction matrix (dimensions 943 x 1,682 with 94.96% sparsity) saved in data/processed/.

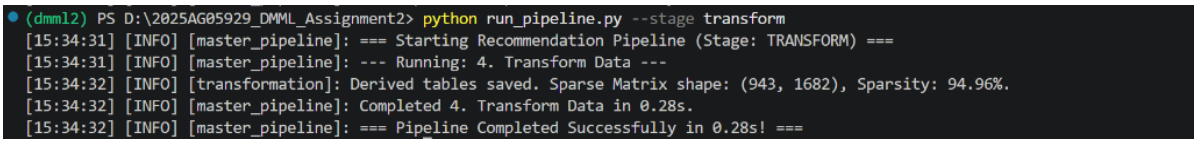


Figure 4: Stage 4 Transformation & Sparse Matrix output (python run_pipeline.py --stage transform)

Stage 5: Feature Management (src/features/feature_store.py)

Builds reusable user, item, and contextual interaction features saved as Parquet tables in data/features/. These features are documented below and used for both training and recommendation ranking.

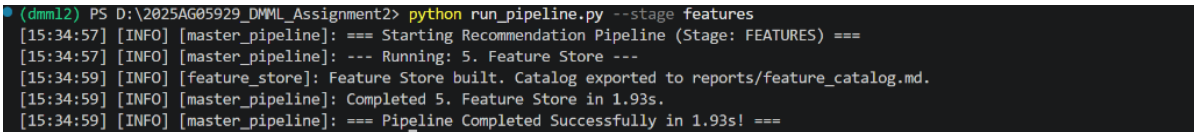


Figure 5: Stage 5 Feature Management output (python run_pipeline.py --stage features)

Feature Name	Entity	Type	Description
user_rating_count	User	INT	Total number of ratings given by user in training set
user_avg_rating	User	FLOAT	Mean rating score given by user
user_preferred_genre	User	STRING	Most frequent high-rated genre for this user
item_popularity_score	Item	FLOAT	Normalized item rating count [0, 1]
interaction_hour	Context	INT	Hour of interaction (time-of-day contextual feature)

2025AG05929

Stage 6: Model Training & MLflow Tracking (src/training/train.py)

I trained two models on the 80,000 training interactions:

1. Baseline Popularity Recommender: Recommends the highest rated, most popular items to all users.
2. Personalized Matrix Factorization (Funk-SVD): Factorizes user-item interactions into 32 latent dimensions, learning user bias b_u and item bias b_i via Stochastic Gradient Descent (SGD) with L2 regularization.

All parameters, metrics, and trained pickle model artifacts are tracked in MLflow using a local SQLite backend (sqlite:///mlflow.db).

```
(dmml2) PS D:\2025AG05929_DMML_Assignment2> python run_pipeline.py --stage train
[15:35:34] [INFO] [master_pipeline]: === Starting Recommendation Pipeline (Stage: TRAIN) ===
[15:35:34] [INFO] [master_pipeline]: --- Running: 6. Train Models ---
[15:35:36] [INFO] [training]: Training Model 1: Baseline Popularity...
[15:35:37] [INFO] [training]: Training Model 2: Personalized Matrix Factorization SVD...
[15:36:05] [INFO] [training]: Models and parameters logged to MLflow.
[15:36:05] [INFO] [master_pipeline]: Completed 6. Train Models in 30.69s.
[15:36:05] [INFO] [master_pipeline]: === Pipeline Completed Successfully in 30.69s! ===
```

Figure 6: Stage 6 Model Training output (python run_pipeline.py --stage train)

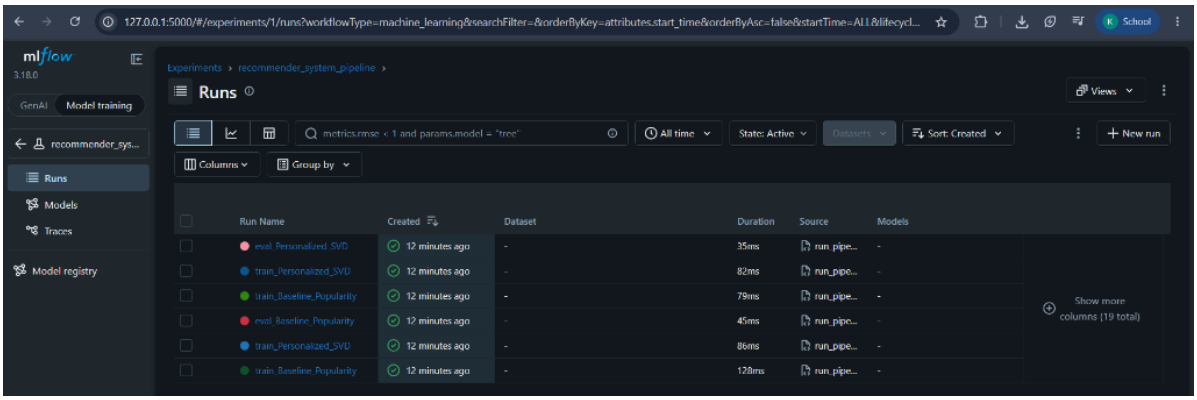


Figure 7: MLflow UI Dashboard showing logged runs, parameters, and models (http://localhost:5000)

Stage 7: Model Evaluation (src/evaluation/evaluate.py)

Both models were evaluated on the held-out test split of 20,000 interactions (relevance threshold r >= 3.5). Evaluation results were saved to reports/model_comparison.csv and reports/evaluation_results.md.

```
(dmml2) PS D:\2025AG05929_DMML_Assignment2> python run_pipeline.py --stage evaluate
[15:49:21] [INFO] [master_pipeline]: === Starting Recommendation Pipeline (Stage: EVALUATE) ===
[15:49:21] [INFO] [master_pipeline]: --- Running: 7. Evaluate Models ---
[15:49:27] [INFO] [evaluation]: Evaluation completed. Results saved in reports/evaluation_results.md.
[15:49:27] [INFO] [master_pipeline]: Completed 7. Evaluate Models in 5.56s.
[15:49:27] [INFO] [master_pipeline]: === Pipeline Completed Successfully in 5.56s! ===
```

Figure 8: Stage 7 Model Evaluation output (python run_pipeline.py --stage evaluate)

Model	RMSE (↓)	MAE (↓)	Prec@5	Prec@10	NDCG@10	Coverage	Diversity
Baseline Popularity	1.1191	0.9478	0.2786	0.2466	0.2719	4.52%	0.8097
Personalized SVD	**1.1006**	**0.9236**	0.0772	0.0872	0.1020	3.92%	0.8008

Findings: The Personalized SVD model achieves lower rating prediction error (RMSE 1.1006 vs 1.1191, MAE 0.9236 vs 0.9478) because it learns personalized user and item biases. The baseline popularity recommender scores higher on precision because popular blockbusters appear frequently in the test set.

Stage 8: Orchestration & Airflow DAG (run_pipeline.py & dags/)

The complete pipeline can be automated in two ways:
1. Python master script: 'python run_pipeline.py --stage all' runs all 7 stages sequentially in ~54 seconds.
2. Apache Airflow DAG: 'dags/recommendation_pipeline_dag.py' defines task dependencies (t1 >> t2 >> t3 >> t4 >> t5 >> t6 >> t7) for scheduled execution.

recommendation_pipeline_dag.py 2

dags > recommendation_pipeline_dag.py > ...

```python
with DAG(
    dag_id="recommendation_pipeline_dag",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule_interval="@daily",
    catchup=False,
) as dag:

    t1 = PythonOperator(task_id="ingest", python_callable=ingest_data)
    t2 = PythonOperator(task_id="validate", python_callable=validate_data)
    t3 = PythonOperator(task_id="prepare", python_callable=prepare_data)
    t4 = PythonOperator(task_id="transform", python_callable=transform_data)
    t5 = PythonOperator(task_id="features", python_callable=generate_features)
    t6 = PythonOperator(task_id="train", python_callable=train_and_track_models)
    t7 = PythonOperator(task_id="evaluate", python_callable=evaluate_and_compare_mod

    t1 >> t2 >> t3 >> t4 >> t5 >> t6 >> t7
```

Figure 9: Apache Airflow DAG code defining task dependencies

2025AG05929

3. Design Decisions & Limitations

Key Decisions:

- Chronological 80/20 Split: Used a time-based cutoff instead of random splitting to prevent data leakage.
- Data Quarantine: Invalid ratings and timestamps are saved to data/errors/ rather than silently dropped.
- Parquet Storage: Used snappy-compressed Parquet files for faster I/O compared to raw CSVs.
- MLflow SQLite: Used sqlite:///mlflow.db for lightweight, reliable local experiment tracking.

Limitations:

- Cold-Start: New users with no ratings fallback to the popularity model.
- Implicit Feedback: Currently trained only on explicit 1-5 ratings; incorporating watch percentage and clicks would improve ranking.